Open-World Pedestrian Crowd Performance Research

Executive Summary

对开放世界街道人群而言，官方引擎文档与经典 crowd 论文的共识非常一致：不要把“人多”直接实现成“成千上万个完整 NPC”。更可持续的方案是把 crowd 拆成多层表示与多层仿真，只让极少数、最接近玩家且最有交互价值的个体升级为完整 agent，其余层级则走 lane-following、低频更新、批量渲染和 streaming-aware 的预算控制 [1][2][3][4][5][6][9][10][11]。

对 `godot_citys` 这个已经被 `16.67ms/frame` 红线约束住的 `70km x 70km` low poly 城市底盘来说，`v6` 最现实的方向不是“全城行人 AI”，而是“从已有 road graph 派生 sidewalk/crossing graph，再构建 Tier 0-3 的分层 pedestrian system”。换句话说，先做可信的街道氛围，再做少量近场反应，而不是把性能债重新引回 streaming 热路径 [4][5][6][7][8][12]。

Key Findings

- **大世界 crowd 的第一原则是分层，而不是人人平权**：经典 crowd LOD 研究明确指出，几何、动作和行为都应按“与玩家的相关性”分层，低重要度个体应使用更廉价的近似表示 [9][10]。
- **Godot 的大规模可见人群更适合走 batched representation，而不是满地 `Node3D`**：`MultiMesh` 能把成千上万实例压成极少 draw call，但没有 per-instance frustum culling，因此必须按空间页或 band 切分，不能把整城人群塞进一个 `MultiMesh` [1]。
- **NavigationAgent/avoidance 只能留给极小的近场集合**：Godot 官方文档明确提醒避免无节制使用 avoidance、不要频繁重算路径，也不要把动态 obstacle 当作每帧都在移动的高频对象 [4][5][6]。
- **行业成熟做法更接近“lane graph + zone graph + data-oriented crowd”**：Unreal 的 Mass/Mass Avoidance 方向，本质上也是把 crowd 建模为数据片段、局部 steering 和邻域处理，而不是“每个人都是一整套重型 AI Actor” [7][8]。
- **对本项目而言，sidewalk/crossing graph 比“先摆人”更重要**：如果没有从 `road_graph` 派生出的行人 lane graph，最终只会回到 v2 以前那种 chunk 内独立随机摆放的拼接感，既不自然，也无法做连续 streaming [12]。
- **性能护栏必须在设计阶段就写死**：当前仓库已经把“每完成一个里程碑都必须重新 profiling”写进纪律；行人系统必须继承这条纪律，否则极易在“看起来有人了”之后重新把帧耗打回失控区间 [12]。

Detailed Analysis

1. 表示层：大量人群首先是“批量可见性问题”

Godot 官方对 `MultiMesh` 的描述很直接：一份 `MultiMesh` 可以一次性画出大量实例，适合成千上万重复对象，但缺点是没有实例级别的 frustum culling，只能“整份可见”或“整份不可见” [1]。这意味着如果要在城市街道里画大量行人，正确做法不是“一个全局行人 `MultiMesh`”，而是把行人实例按 chunk、page 或 distance band 拆开，再与 visibility ranges 和 occlusion culling 组合使用 [1][2][3]。

这套思路和当前项目的城市底盘是兼容的。`v5` 已经把 terrain / road surface 的热点压下去了，说明 `godot_citys` 现阶段最大的敌人不是“绝对没有性能”，而是“每次新增系统都不能重新制造主线程热路径” [12]。把中远景 pedestrian 设计成 batched representation，天然比“数百上千个 `CharacterBody3D` 同时常驻”安全。

对本仓库的建议是：中远景 pedestrian 只保留位置、朝向、速度相位、archetype 和 route progress 这些最小数据；近景才实例化更重的节点。这一建议是基于 [1][2][3][12] 的工程推断，而不是某一篇论文的逐字要求。

2. 导航层：不要把全城行人直接做成 `NavigationAgent3D`

Godot 的导航性能文档强调两件事：第一，大世界要尽量减少高复杂度导航数据与高频路径重算；第二，avoidance 并不是“白送的”，它本身就是每帧要持续计算的成本 [4][5][6]。`NavigationObstacle` 文档还明确提到，如果 obstacle 不用于 avoidance，应关闭相关能力；对静态 obstacle 来说，频繁移动和重建也是昂贵的 [5][6]。

这对 `v6` 的约束非常明确：不能把“街道人群”翻译成“每个 pedestrian 一个 `NavigationAgent3D`，全都开 avoidance，再每帧追目标”。这种实现路径虽然最直觉，但几乎注定会把当前刚守住的红线打穿。

更现实的做法是：

1. 先从 `road_graph` 和 `block` 数据派生出 `sidewalk/crossing lane graph`。
2. 大多数 pedestrian 只做 graph lane-following，不做全局 navmesh 查询。
3. 只有玩家附近极小半径内的少量 reactive pedestrians，才允许进入更贵的 steering / avoidance 层。

这与 Unreal 的 Mass Avoidance 路线也是一致的：它关注的是局部 steering 与邻域碰撞规避，而不是让所有实体都走一套传统“厚 Actor + 厚导航”的路径 [7][8]。

3. 仿真层：经典论文的核心不是“更真实”，而是“按重要度降本”

关于 crowd 的经典 LOD 论文给出的方向其实非常务实。Hodgins 与 Carlson 很早就提出，群体角色可以按重要度切换到底层仿真复杂度不同的表示：高重要度个体保留动态仿真，低重要度个体改用更便宜的近似 [9]。O'Sullivan 等人的 crowd/group LOD 框架则进一步把这个思想扩展到几何、动作、自然行为和社交行为本身 [10]。

这类研究对 `godot_citys` 的直接启示是：

- Tier 之间不应该只切 mesh 细节，还应该切更新频率、行为复杂度和动画 fidelity。
- “离玩家远”与“和玩家无交互价值”应该直接等价为更廉价的仿真层。
- 如果一个 pedestrian 的存在只服务于“让街上看起来有人”，那它就不应占用完整 agent 的预算。

`Continuum Crowds` 提供了另一类重要视角：把大群体视为连续流场，从宏观密度与方向场角度做高密度 crowd motion [11]。这对“超大规模总体流向”很有启发，但对本项目当前阶段来说，直接把 v6 建成 continuum system 并不现实。更合理的用法是把它当成远期参考：未来如果需要整城通勤流量或事件人流，可把它作为 Tier 0 的统计层思路，而不是 v6 首轮实现。

4. 对 `godot_citys` 的推荐架构：Tier 0-3 分层 pedestrian system

基于 [1][4][7][9][10][12]，对本仓库最稳妥的推荐方案如下：

Tier	表示	推荐半径/范围	推荐上限	更新方式	说明
Tier 0	纯数据 occupancy / lane reservation	活跃 `5x5` chunk 全窗口	数据层为主	低频或事件驱动	不生成 3D actor，只为后续 lane 占用、density 和 spawn 提供真相层
Tier 1	batched ambient visuals	约 `45m - 160m`	`<= 768` 可见实例	`2Hz - 4Hz`	用 `MultiMesh` / 合批表示中远景行人
Tier 2	lightweight local agents	约 `15m - 45m`	`<= 96`	`5Hz - 10Hz`	有更稳定的移动和朝向，但仍不做全量 avoidance
Tier 3	reactive near-	`<= 35m`	`<= 24`	逐帧或高频	才允许简单 sidestep、panic、让路、受枪火惊扰

Tier	表示	推荐半径/范围	推荐上限	更新方式	说明
	player agents				

上表中的数值是针对本仓库当前性能基线做的工程建议，是基于 [1][4][9][12] 的推断。它们不是行业标准常数，但它们提供了一个足够硬的初始预算口径，方便后续用 profiling 去证伪或收紧。

5. 对 `v6` 的实现顺序建议：先建 lane graph，再做人

如果按工程收益排序，`v6` 的正确顺序应当是：

1. **行人世界模型与密度配置**：先有可查询、可复现、可按 chunk 派生的 ped data。
2. `sidewalk/crossing lane graph`：先把“人该走哪里”变成世界数据，而不是先实例化“人”。
3. **Tier 1 / Tier 2 基础表示**：先把街道氛围做出来，并验证 continuity 与 spawn/despawn。
4. **Tier 3 近场反应**：只给最少量个体加上更贵的本地行为。
5. **红线收口与 observability**：把 crowd 相关 profiling 指标、debug overlay、minimap 调试层补齐。

这个顺序的核心原因是：lane graph 和 world model 是街道人群的骨架；没有骨架，后面看到的只会是“随机摆人”，既不自然，也无法做稳定的性能优化。

6. 对性能风险的具体判断

从当前项目状态出发，真正危险的点主要有四个：

1. **Node 数量膨胀**：如果 Tier 1 也直接实例化节点，scene tree、transform 更新和可见性判断都会变重。
2. **avoidance 范围失控**：如果 Tier 2/Tier 3 数量没有硬上限，局部 reactive 行为很快会扩散成全局成本。
3. **路径查询过于频繁**：如果每个 pedestrian 持续重算目标或重建路径，CPU 会被 crowd update 吃掉。
4. **streaming 与 crowd 解耦失败**：如果 chunk unload/reload 时需要重新完整生成整组 pedestrians，就会把首访成本重新拉高。

因此，`v6` 设计时就必须写入以下护栏：

- 默认模式必须是 `pedestrian_mode = lite`；
- 所有 tier 数量都必须有 runtime 可读的硬上限；
- 每个 milestone 完成后必须重跑 fresh profiling；

- profiling 输出必须新增 `crowd_update_avg_usec`、`crowd_spawn_avg_usec`、`crowd_render_commit_avg_usec`、各 tier 数量与 lane graph cache 命中字段。

这些约束不是额外负担，而是让 v6 有机会在当前底盘上真正落地的必要条件。

Areas of Consensus

- 大规模 crowd 系统应同时对**几何、动作、行为和 AI 更新频率**做分层，而不是只对 mesh 做 LOD [9][10]。
- 中远景 crowd 更适合走 **batch / data-oriented** 表示；只有近景才值得升格为更重的 agent [1][7][8]。
- avoidance 与高频路径更新应严格受限，不适合无上限地应用到所有 pedestrian [4][5][6]。
- 大世界 crowd 必须与 streaming / page / chunk 系统绑定，否则首访和回访成本都难以控制 [1][2][12]。

Areas of Debate

- **远景 crowd 该走 `MultiMesh`、billboard impostor 还是更轻的 shader 代理**：Godot 官方对 `MultiMesh` 非常友好，但它的全或无可见性要求更细的空间切分；是否需要更轻的 impostor，要等 v6 的第一轮 profiling 再定 [1]。
- **Tier 2 是否需要真正的 `NavigationAgent3D`**：从性能风险看，能不用就不用；但如果局部路径修正复杂度不足，可能需要在极小子集上引入。这个点应在 M4 前后用数据决策，而不是预设立场 [4][6]。
- **是否需要在 v6 就引入“人群流场”**：`Continuum Crowds` 很适合超大尺度整体流动，但它更像未来的 density layer 方向，不是当前最短路径 [11]。
- **动画 fidelity 的下限设在哪**：low poly 风格允许用更简单的 pose / phase 代理，但“简单到什么程度仍然可信”仍需要项目内人工试玩与 profiling 一起决定 [10][12]。

Sources

[1] Godot Engine. *Optimization using MultiMeshes*. https://docs.godotengine.org/en/latest/tutorials/performance/using_multimesh.html （官方文档，高可信）

[2] Godot Engine. *Visibility ranges (HLOD)*. https://docs.godotengine.org/en/stable/tutorials/3d/visibility_ranges.html （官方文档，高可信）

[3] Godot Engine. *Occlusion culling*. https://docs.godotengine.org/en/stable/tutorials/3d/occlusion_culling.html （官方文档，高可信）

[4] Godot Engine. *Optimizing Navigation Performance*. https://docs.godotengine.org/en/4.1/tutorials/navigation/navigation_optimizing_performance.html （官方文档，高可信）

[5] Godot Engine. *Using NavigationObstacles*. https://docs.godotengine.org/en/4.4/tutorials/navigation/navigation_using_navigationobstacles.html （官方文档，高可信）

[6] Godot Engine. *Thread-safe APIs*. https://docs.godotengine.org/en/latest/tutorials/performance/thread_safe_apis.html ; Godot Engine. *Using multiple threads*. https://docs.godotengine.org/en/stable/tutorials/performance/using_multiple_threads.html （官方文档，高可信）

[7] Epic Games. *Mass Avoidance Overview in Unreal Engine*. https://dev.epicgames.com/documentation/en-us/unreal-engine/mass-avoidance-overview-in-unreal-engine （官方引擎文档，高可信）

[8] Epic Games. *MassEntity API / Framework Overview*. https://dev.epicgames.com/documentation/en-us/unreal-engine/API/Runtime/MassEntity （官方引擎文档，高可信）

[9] Jessica K. Hodgins, Deborah A. Carlson. *Simulation Levels of Detail for Real-time Animation*. Georgia Tech GVU Technical Report, 1996. https://repository.gatech.edu/items/36e25967-ac29-4e66-8d15-404861cc3910 （一手研究资料，高可信）

[10] Carol O'Sullivan et al. *Levels of Detail for Crowds and Groups*. https://www.media.mit.edu/gnl/publications/crowds1.pdf （研究论文/研究小组公开稿，高可信）

[11] Adrien Treuille, Seth Cooper, Zoran Popovic. *Continuum Crowds*. SIGGRAPH 2006. https://grail.cs.washington.edu/projects/crowd-flows/continuum-crowds.pdf （经典论文，高可信）

[12] `godot_citys` 内部基线文档：`docs/research/2026-03-12-fps60-redline-baseline.md`、`docs/plan/v5-index.md` （项目内一手 profiling 证据，高可信）

Gaps and Further Research

- 还没有对 `MultiMesh` 行人使用真实相机、真实 GPU 渲染的 profile 做本项目内 A/B；当前主要结论仍偏向 headless/dummy 与架构层推导。
- 还没有针对“楼梯、坡道、高架人行通道”建立独立 lane graph 规则；v6 首轮更适合先做平面街道与基础 crossing。
- 还没有研究“行人被车辆、敌人、爆炸物影响”的完整行为矩阵；v6 应先把行为约束在玩家附近极小集合。
- 如果未来要做白天/夜晚、通勤潮汐和 district 事件流量，还需要追加一轮关于 crowd scheduling / density field 的专项 research。